Simularea gravitațională a sistemelor plane:
validare kepleriană, conservarea energiei și dinamica multicorp

Georgescu Andrei

29.10.2026

Rezumat

Se prezintă un simulator gravitațional bidimensional realizat în C++, cu integrare kick–drift–kick leapfrog, tratarea coliziunilor prin fuziune și vizualizare în Python. Studiul reunește experimentele cu două, trei, patru și zece corpuri, păstrând distincte rezultatele istorice și calculele reproductibile efectuate pentru această lucrare. Validarea principală compară direct mișcarea a două corpuri cu soluția kepleriană, la aceiași timpi: pe două zile, reducerea pasului de la 5 s la 0,1 s scade eroarea maximă eșantionată a poziției relative de la 0,753 m la 0,312 mm. Ordinul observat este 2,000 pentru energia maximă și 1,991 pentru poziție. O simulare separată cu trei corpuri este comparată cu un integrator DOP853 independent. În experimentul cu zece corpuri pe 100 de ani, variația relativă maximă a energiei ajunge la 80,57, în asociere temporală cu o apropiere strânsă. Rezultatele evidențiază diferența dintre conservarea energetică, precizia traiectoriei și comportamentul unei simulări la întâlniri apropiate.

Cuvinte-cheie: gravitație newtoniană; problema celor două corpuri; leapfrog; velocity Verlet; eroare energetică; soluție kepleriană.

1 Introducere

Simularea unui sistem gravitațional presupune transformarea unui model fizic continuu într-o succesiune de stări discrete. O traiectorie cu aspect orbital nu stabilește singură corectitudinea calculului: aceeași curbă poate ascunde o eroare de fază, o variație a energiei sau o eroare de timp. Din acest motiv, lucrarea tratează împreună modelul, implementarea și verificarea cantitativă.

Proiectul folosește interacțiuni newtoniene directe între toate perechile de corpuri. Integrarea este realizată în C++, iar pozițiile sunt salvate într-un fișier binar pentru explorare în Python. Corpurile au masă și rază, se deplasează în plan și pot fuziona la contact. În versiunile analizate aici, toate corpurile active sunt integrate, inclusiv corpul cel mai masiv.

Obiectivul central este măsurarea erorii de traiectorie pentru cazul în care există o soluție de referință analitică: două corpuri izolate. Sistemele cu mai multe corpuri sunt analizate separat. Referința kepleriană nu este prezentată drept soluție exactă a problemei generale cu trei sau zece corpuri.

2 Materiale și organizarea experimentelor

2.1 Surse și proveniență

Analiza se bazează pe fișierele `main.cpp`, `star.h`, `star.cpp`, `graph.py`, intrările numerice și fișierele binare păstrate. Au fost reunite valorile comunicate în conversațiile despre documentație și în conversația *Find what's in this file*. Fiecare rezultat este încadrat într-una dintre categoriile: valoare comunicată de autor, măsurătoare dintr-un binar disponibil sau reluare controlată a algoritmului.

Fișierul `simulation.bin` a fost reutilizat pentru experimente diferite. În consecință, numele său nu identifică un experiment. Pentru această analiză s-au folosit copii stabile: un caz cu trei corpuri, 2 zile și pas de 0,5 s, respectiv un caz cu zece corpuri, 100 ani și pas de 3600 s. Binarul istoric al fiecărei rulări cu două corpuri nu mai este disponibil separat; pentru acestea s-a refăcut experimentul din condițiile inițiale recuperate.

2.2 Condițiile inițiale ale cazului K2

Intrarea comună tuturor reluărilor K2 diferă numai prin pasul temporal:

```
2 172800 0.1
1e15 100 -5000 0 0 -0.65
5e14 80 10000 0 0 1.30
```

Tabelul 1: Experimentele centrale ale lucrării. Un an în cazul N10 are 365 de zile.

Caz	Corpuri	Interval / pas	Baza comparației
K2	2	2 zile; $h = 5, 2, 1, 0{,}5, 0{,}1$ s	Reluări C++; soluția Kepler
N3	3	2 zile; $h = 0{,}5$ s	Binar; DOP853 independent
C4	4	5000 s; $h = 0{,}1$ s	Eveniment de fuziune arhivat
N10	10	100 ani; $h = 3600$ s	Binar complet; energie reconstruită

Ordinea câmpurilor pentru un corp este m, R, x, y, v_x, v_y, în unități SI. Masele sunt $m_1 = 10^{15}$ kg și $m_2 = 5 \times 10^{14}$ kg. Separarea inițială este 15 km; viteza relativă este 1,95 m/s și este perpendiculară pe vectorul de separare. Centrul de masă și impulsul inițial sunt nule. Suma razelor este 180 m, mult sub distanța la pericentru.

3 Modelul matematic

3.1 Forțele și ecuațiile de mișcare

Pentru N corpuri cu centre distincte,

$$\mathbf{F}_{ij} = G m_i m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{\|\mathbf{r}_j - \mathbf{r}_i\|^3}, \qquad G = 6{,}67430 \times 10^{-11}\ \mathrm{m^3\,kg^{-1}\,s^{-2}}, \tag{1}$$

$$\dot{\mathbf{r}}_i = \mathbf{v}_i, \qquad \dot{\mathbf{v}}_i = \mathbf{a}_i = \frac{1}{m_i} \sum_{j \neq i} \mathbf{F}_{ij}. \tag{2}$$

Programul calculează numai perechile $i < j$ și aplică simultan forța opusă asupra corpului j. Pentru fiecare evaluare sunt necesare $N(N-1)/2$ perechi: complexitate $O(N^2)$, fără aproximări de tip arbore sau rețea.

Mișcarea este plană, însă interacțiunea este legea gravitației newtoniene în $1/r^2$, restrânsă la configurații coplanare. Razele sunt folosite la detecția contactului; nu modifică expresia forței înaintea coliziunii.

3.2 Energia, impulsul și centrul de masă

Energia mecanică implementată este

$$E = K + U = \sum_i \frac{m_i}{2} \|\mathbf{v}_i\|^2 - \sum_{i<j} \frac{G m_i m_j}{r_{ij}}, \qquad r_{ij} = \|\mathbf{r}_j - \mathbf{r}_i\|. \tag{3}$$

Suma potențială folosește o singură contribuție pentru fiecare pereche. Energia inițială negativă a unui sistem nu înseamnă că fiecare corp este individual legat pentru totdeauna.

$$\mathbf{P} = \sum_i m_i \mathbf{v}_i, \quad \mathbf{R}_{\mathrm{CM}} = \frac{\sum_i m_i \mathbf{r}_i}{\sum_i m_i}, \quad L_z = \sum_i m_i (x_i v_{y,i} - y_i v_{x,i}). \tag{4}$$

Pentru un sistem izolat fără fuziuni, E, $\mathbf{P}$ și L_z sunt constante, iar $\mathbf{R}_{\mathrm{CM}}(t) = \mathbf{R}_{\mathrm{CM}}(0) + t\mathbf{P}(0)/\sum_i m_i$. Această ultimă formulă, nu ipoteza că centrul de masă ar fi mereu fix, este folosită în analiza N10.

3.3 Coliziuni și fuziuni

Contactul este detectat prin $r_{AB} \leq R_A + R_B$. Fuziunea folosește

$$M = m_A + m_B, \qquad \mathbf{R} = \frac{m_A \mathbf{r}_A + m_B \mathbf{r}_B}{M}, \tag{5}$$

$$\mathbf{V} = \frac{m_A \mathbf{v}_A + m_B \mathbf{v}_B}{M}, \qquad R_{\mathrm{nou}} = (R_A^3 + R_B^3)^{1/3}. \tag{6}$$

Se conservă masa și impulsul liniar, iar razele corespund adunării volumelor sferice. Diferența energiei cinetice de translație este

$$\Delta K = -\frac{1}{2} \frac{m_A m_B}{m_A + m_B} \|\mathbf{v}_A - \mathbf{v}_B\|^2. \tag{7}$$

Aceasta nu reprezintă variația întregii energii din (3): după fuziune dispare potențialul perechii și se modifică interacțiunile cu celelalte corpuri. Modelul nu stochează energia internă sau rotația remanentului. Din acest motiv, un salt la coliziune nu se interpretează în același mod ca abaterea energetică a unei orbite fără contact.

4 Metoda numerică și implementarea

4.1 Schema kick–drift–kick

Pentru un pas fix h, fără fuziune în interiorul lui,

$$\mathbf{v}_i^{n+1/2} = \mathbf{v}_i^n + \tfrac{h}{2}\mathbf{a}_i(\mathbf{r}^n), \tag{8}$$

$$\mathbf{r}_i^{n+1} = \mathbf{r}_i^n + h\mathbf{v}_i^{n+1/2}, \tag{9}$$

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^{n+1/2} + \tfrac{h}{2}\mathbf{a}_i(\mathbf{r}^{n+1}). \tag{10}$$

Aceasta este forma kick–drift–kick a leapfrog, echivalentă cu velocity Verlet pentru forțe dependente de poziție. Viteza de ieșire este disponibilă la același timp cu poziția; etapa intermediară folosește viteze la jumătate de pas.

Schema are ordin global doi și este simetrică în timp și simplectică în regimul conservativ cu pas fix. Aceste proprietăți explică buna conservare energetică pe intervale lungi atunci când mișcarea este rezolvată suficient de fin [1]. Nu presupun conservarea exactă a energiei la fiecare iterație. Detecția și aplicarea fuziunilor constituie evenimente discrete suplimentare.

Listarea 1: Nucleul calculului unei perechi în set_force.

```
const double magnitude =
    G * objects[i].get_mass() * objects[j].get_mass()
    / distance / distance;
const vector_2d force = displacement * (magnitude / distance);
forces[i] = forces[i] + force;
forces[j] = forces[j] + force * (-1.0);
```

`vector_2d` stochează componentele și oferă adunare și înmulțire cu scalar. Înmulțirea dintre doi vectori definită în clasă este pe componente, nu produs scalar. `celestial_object` încapsulează masa, raza, poziția, viteza și accelerația. `set_force` rezolvă contactele, resetează forțele și acumulează perechile; `total_energy` implementează (3).

4.2 Ordinea operațiilor și protocolul de ieșire

În bucla principală, energia și pozițiile sunt evaluate înaintea actualizării stării. După primul semipas de viteză, toate pozițiile sunt actualizate, apoi se recalculează toate forțele și se completează vitezele. Separarea buclelor păstrează sincronizarea corpurilor.

Antetul binar conține N_0, T și h, iar un cadru conține timpul, numărul activ și perechile (x, y). În platforma analizată, dimensiunile sunt:

$$B_{\text{antet}} = 4 + 8 + 8 = 20 \text{ octeți}, \qquad B_{\text{cadru}} = 8 + 4 + 16N_{\text{activ}}. \tag{11}$$

Python citește explicit little-endian: `<idd` pentru antet și `<di` pentru cadrul temporal. Câmpurile sunt scrise separat; nu se salvează direct structura internă a obiectului. Binarul nu stochează masele, razele sau vitezele.

Vizualizatorul verifică blocurile incomplete, monotonia timpilor, valorile finite și numărul activ. Interfața permite Play/Pause, navigare cu săgețile, cursor temporal și salt la cel mai apropiat timp salvat. După fuziune, indicii se pot modifica; traseele sunt restartate la schimbarea numărului de corpuri. Comanda `--save` produce o figură statică a ultimei stări.

4.3 Reluările controlate pentru validare

Reluările K2 folosesc funcțiile originale din `star.cpp`, într-un program separat de măsurare. Nu s-au modificat sursele din directorul proiectului utilizatorului. Compilarea reluărilor a folosit MSVC x64, C++17 și optimizarea `/O2`, în precizie `double`, fără activarea modului de aritmetică rapidă. Antetul standard `type_traits` a fost inclus la compilare.

Programul măsoară energia la fiecare pas și salvează pozițiile și vitezele la intervale de 5 s, respectiv de 4 s pentru $h = 2$ s. Ultima stare este inclusă. Măsurătorile maxime de poziție se referă la aceste eșantioane, nu la toate stările

intermediare. Ca în programul furnizat, timpul se acumulează prin adunări repetate; comparația analitică folosește timpul efectiv al fiecărui rând salvat.

5 Soluția analitică pentru două corpuri

5.1 Reducerea la mișcarea relativă

Definind $\mathbf{r} = \mathbf{r}_2 - \mathbf{r}_1$, $\mathbf{v} = \mathbf{v}_2 - \mathbf{v}_1$ și $\mu = G(m_1 + m_2)$,

$$\ddot{\mathbf{r}} = -\mu \frac{\mathbf{r}}{r^3}, \quad \varepsilon = \frac{v^2}{2} - \frac{\mu}{r}, \quad a = -\frac{\mu}{2\varepsilon}, \quad \mathbf{e} = \frac{(v^2 - \mu/r)\mathbf{r} - (\mathbf{r}\cdot\mathbf{v})\mathbf{v}}{\mu}. \tag{12}$$

Pentru cazul K2 se obțin $\mu = 100114{,}5$ m^3/s^2, $a = 10487{,}476561$ m și $e = 0{,}4302773325$. Separarea inițială este apocentrul. Parametrii orbitei relative sunt

$$r_p = 5974{,}953122 \text{ m}, \quad r_a = 15000 \text{ m}, \quad P = 2\pi\sqrt{\frac{a^3}{\mu}} = 21327{,}398405 \text{ s}. \tag{13}$$

Intervalul de două zile cuprinde aproximativ 8,102 perioade. Centrele corpurilor nu ating pragul geometric de 180 m.

5.2 Propagarea kepleriană la același timp

Anomalia medie și anomalia excentrică sunt legate prin ecuația Kepler [2]:

$$M(t) = M_0 + nt, \quad n = \sqrt{\mu/a^3}, \quad u - e\sin u = M, \qquad M_0 = \pi. \tag{14}$$

Simbolul u evită confuzia cu energia E. Ecuația scalară este rezolvată prin Newton, după reducerea anomaliei medii modulo 2π:

$$u_{k+1} = u_k - \frac{u_k - e\sin u_k - M}{1 - e\cos u_k}. \tag{15}$$

Reziduul maxim măsurat este $4{,}44 \times 10^{-16}$ rad. În baza pericentrului $\hat{\mathbf{p}} = (-1, 0)$ și $\hat{\mathbf{q}} = (0, -1)$,

$$\mathbf{r}_{\text{Kep}}(t) = a(\cos u - e)\hat{\mathbf{p}} + a\sqrt{1 - e^2}\sin u\hat{\mathbf{q}}. \tag{16}$$

Pozițiile individuale rezultă din mișcarea centrului de masă:

$$\mathbf{r}_1 = \mathbf{R}_{\text{CM}} - \frac{m_2}{m_1 + m_2}\mathbf{r}, \qquad \mathbf{r}_2 = \mathbf{R}_{\text{CM}} + \frac{m_1}{m_1 + m_2}\mathbf{r}. \tag{17}$$

Aceasta este soluția analitică kepleriană a mișcării, evaluată numeric printr-o ecuație scalară; referința nu integrează pas cu pas aceleași ecuații cu leapfrog.

Ca verificare independentă, propagarea relativă DOP853 a fost comparată cu (16) la 2001 de timpi. Toleranțele relative 10^{-11} și 10^{-13} au dat diferențe maxime de 0,107 mm, respectiv 0,00129 mm. Reducerea diferenței susține consistența evaluării kepleriene la scara erorilor raportate.

6 Definirea erorilor și rezultate pentru K2

6.1 Energia maximă și abaterea absolută maximă

Cele două valori afișate de program măsoară lucruri diferite:

$$\Delta E_+ = \max_n E_n - E_0, \qquad \Delta E_{\max} = \max_n |E_n - E_0|, \tag{18}$$

$$\epsilon_{E,+} = \frac{\Delta E_+}{|E_0|}, \qquad \epsilon_{E,\max} = \frac{\Delta E_{\max}}{|E_0|}. \tag{19}$$

Variabila `max_energy` este inițializată cu E_0, deci $\Delta E_+ \geq 0$. Pentru o energie negativă, energia maximă este cea mai apropiată de zero, nu cea cu modulul maxim. `energy error` urmărește abaterea pozitivă, iar `max energy error` urmărește ambele sensuri.

În cazul K2, $K_0 = 6{,}3375 \times 10^{14}$ J, $U_0 = -2{,}2247666667 \times 10^{15}$ J și $E_0 = -1{,}5910166667 \times 10^{15}$ J. Valorile istorice comunicate de autor sunt:

	Seria A	Seria B
ΔE_+ (J)	43 741 097,75	18 345,00
$\Delta E_{\max}$ (J)	5 070 864 995,75	2 028 292,25
$\epsilon_{E,\max}$	$3{,}18718534 \times 10^{-6}$	$1{,}27484035 \times 10^{-9}$

Raportul abaterilor absolute este 2500,066. Rulările controlate cu $h = 5$ s și $h = 0{,}1$ s dau valori foarte apropiate de aceste două serii, cu diferențe sub 0,005% pentru abaterea absolută. Această corespondență este compatibilă cu un raport al pașilor de 50, dar nu dovedește retrospectiv setările originale ale seriei A. Condițiile inițiale sunt aceleași; compilatorul și istoricul complet al rulărilor inițiale nu sunt cunoscute.

6.2 Eroarea poziției, fazei și formei orbitei

Comparația principală este sincronă:

$$\delta_r(t_n) = \|\mathbf{r}_{\text{num}}(t_n) - \mathbf{r}_{\text{Kep}}(t_n)\|, \qquad \delta^{(s)}_{r,\max} = \max_{n \in \mathcal{S}} \delta_r(t_n). \tag{20}$$

Mulțimea $\mathcal{S}$ conține stările salvate pentru analiză. Exponentul (s) marchează explicit maximul eșantionat. Datorită condițiilor baricentrice, erorile individuale sunt $\delta_{r,1} = \delta_r/3$ și $\delta_{r,2} = 2\delta_r/3$; în reluări, centrul de masă este nul la precizia afișată.

Pentru a separa geometria de sincronizare s-au calculat și

$$\delta_\phi = \operatorname{unwrap}(\arg \mathbf{r}_{\text{num}}) - \operatorname{unwrap}(\arg \mathbf{r}_{\text{Kep}}), \qquad \delta_{\text{formă}} = r_{\text{num}} - \frac{a(1-e^2)}{1 + e\cos\nu_{\text{num}}}. \tag{21}$$

Ultima mărime este o abatere radială la aceeași direcție, nu distanța euclidiană minimă până la elipsă. O figură în care orbitele se suprapun poate ascunde o eroare de fază; de aceea sunt prezentate și curbele erorilor.

Tabelul 2: Reluări K2: energie măsurată la fiecare pas și erori de poziție relativă la timpii eșantionați. $\delta_{r,f}$ este eroarea la ultima stare a fiecărei rulări.

h (s)	$\Delta E_{\max}$ (J)	$\epsilon_{E,\max}$	$\delta^{(s)}_{r,\max}$ (m)	$\delta_{r,f}$ (m)
5	$5{,}071 \times 10^{9}$	$3{,}187 \times 10^{-6}$	$7{,}530 \times 10^{-1}$	$7{,}273 \times 10^{-1}$
2	$8{,}113 \times 10^{8}$	$5{,}100 \times 10^{-7}$	$1{,}205 \times 10^{-1}$	$1{,}164 \times 10^{-1}$
1	$2{,}028 \times 10^{8}$	$1{,}275 \times 10^{-7}$	$3{,}012 \times 10^{-2}$	$2{,}909 \times 10^{-2}$
0.5	$5{,}071 \times 10^{7}$	$3{,}187 \times 10^{-8}$	$7{,}530 \times 10^{-3}$	$7{,}273 \times 10^{-3}$
0.1	$2{,}028 \times 10^{6}$	$1{,}275 \times 10^{-9}$	$3{,}119 \times 10^{-4}$	$3{,}022 \times 10^{-4}$

Pentru $h = 5$ s, maximul eșantionat al erorii relative este 0,7530 m, iar erorile maxime individuale sunt 0,2510 m și 0,5020 m. Pentru $h = 0{,}1$ s, valorile individuale sunt 0,1040 mm și 0,2079 mm. Eroarea relativă de poziție la ultimul timp este 0,3022 mm; maximul eșantionat este 0,3119 mm.

Abaterea radială maximă a formei este 0,2860 m la $h = 5$ s și 0,1144 mm la $h = 0{,}1$ s. Erorile de fază maxime sunt $5{,}0203 \times 10^{-5}$ rad și $2{,}0795 \times 10^{-8}$ rad. Momentul cinetic relativ se conservă în reluări la aproximativ 10^{-13} sau mai bine. Conservarea sa nu implică anularea erorii de fază.

6.3 Convergența observată și efectul timpului acumulat

Un model $\delta(h) \approx Ch^p$ permite estimarea ordinului prin regresie liniară a $\log\delta$ în funcție de $\log h$. Pentru cele cinci reluări, $p_E = 2{,}00002$ și $p_r = 1{,}99136$. Raportul abaterilor energetice pentru 5 s și 0,1 s este 2500,166, apropiat de $(5/0{,}1)^2 = 2500$. Rezultatul corespunde ordinului doi al schemei.

În rularea cu pas de 0,1 s, ultima stare este la $172799{,}90000557038$ s, nu la 172800 s. Comparația folosește acest timp efectiv. Diferența acumulată față de indicele înmulțit cu pasul este de ordinul microsecundelor; la erori submilimetrice ea devine observabilă. Rulările cu pași reprezentabili exact în binar includ capătul de interval. Diferențele mici ale ordinului de poziție față de 2 sunt compatibile cu eșantionarea și cu aritmetica finită.

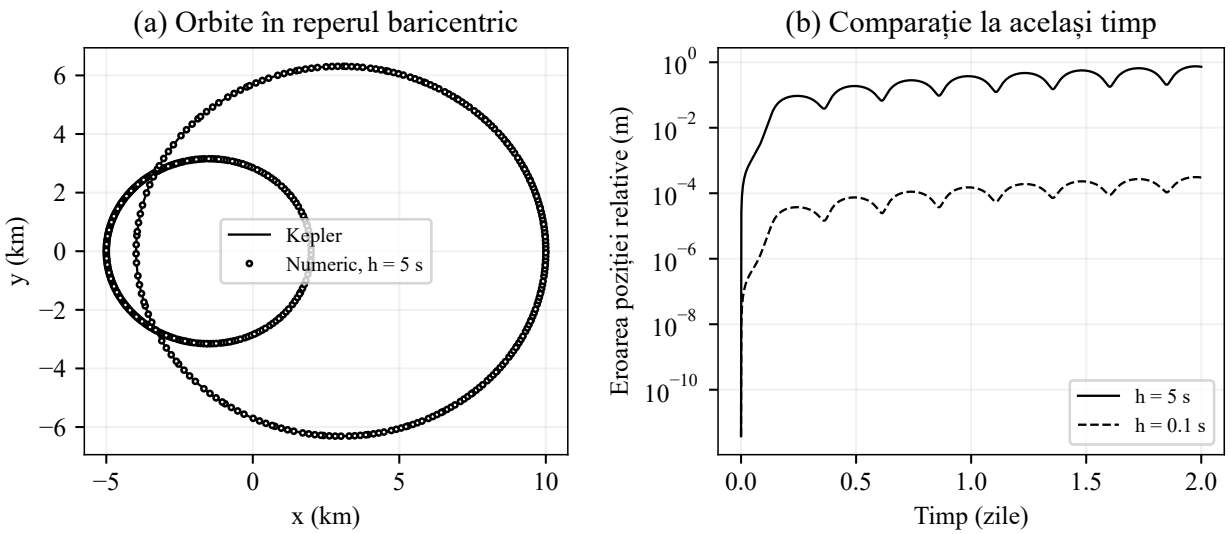


Figura 1: Cazul K2. Stânga: soluția Kepler și puncte numerice pentru $h = 5$ s, în reper baricentric. Dreapta: eroarea poziției relative la același timp; scara logaritmică face vizibile diferențele ascunse de suprapunerea orbitelor.

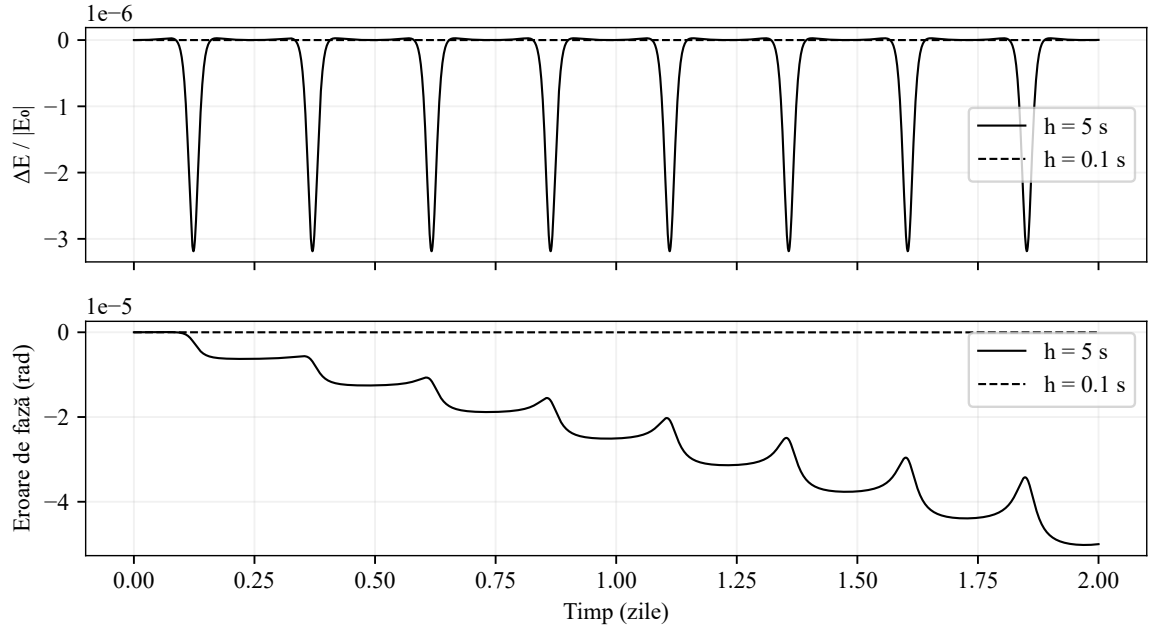


Figura 2: Abaterea energetică relativă și eroarea de fază pentru două valori ale pasului. Energia oscilează, în timp ce eroarea de fază poate acumula o componentă sistematică. Curba pentru pasul mic este apropiată de zero pe scara comună.

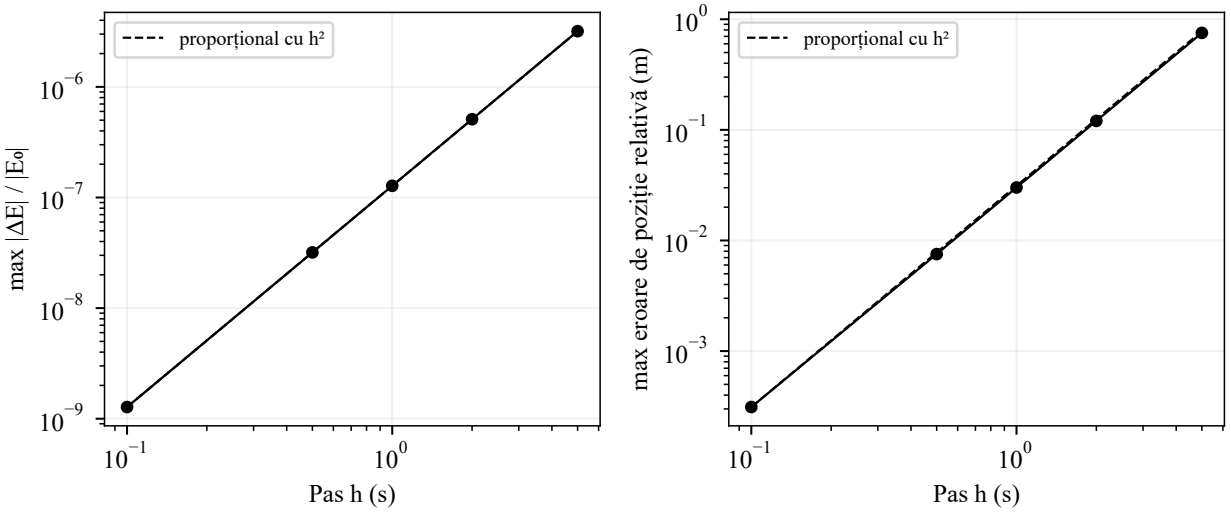


Figura 3: Convergența erorii energetice și a erorii maxime eșantionate de poziție relativă. Liniile întrerupte sunt ghiduri proporționale cu h^2, normalizate la rularea cu pasul cel mai mic.

7 Extinderea la trei corpuri și la fuziuni

7.1 Cazul N3 și referința numerică independentă

În cazul N3, masele sunt 10^{15}, 10^{10} și 3×10^{10} kg, iar intrarea este reprodusă în anexe. Binarul conține 345601 cadre la pas de 0,5 s, cu numărul activ constant 3, timpi uniformi și coordonate finite. Energia inițială este $-91886372572{,}5$ J.

Spre deosebire de K2, soluția de referință N3 este numerică. S-au integrat toate cele trei corpuri prin DOP853, cu variabile adimensionalizate prin $L = 10^4$ m și $S = 10^4$ s, folosind toleranțe relative 10^{-12} și 3×10^{-14}; toleranțele absolute au fost de o mie de ori mai mici. Referințele sunt comparate cu binarul la 17281 de stări, din 10 în 10 s, inclusiv ultima stare.

Diferența maximă dintre cele două referințe este 0,0312 mm. Față de referința mai strictă, eroarea maximă eșantionată este 13,20 mm pentru corpul interior și 0,2246 mm pentru corpul exterior; corpul masiv are o diferență maximă de $1{,}25 \times 10^{-7}$ m. La final, erorile sunt 12,10 mm și 0,2160 mm pentru cele două corpuri mici. Prin urmare, acordul referințelor este sub scara diferențelor de traiectorie măsurate, fără a fi o certificare matematică a erorii DOP853.

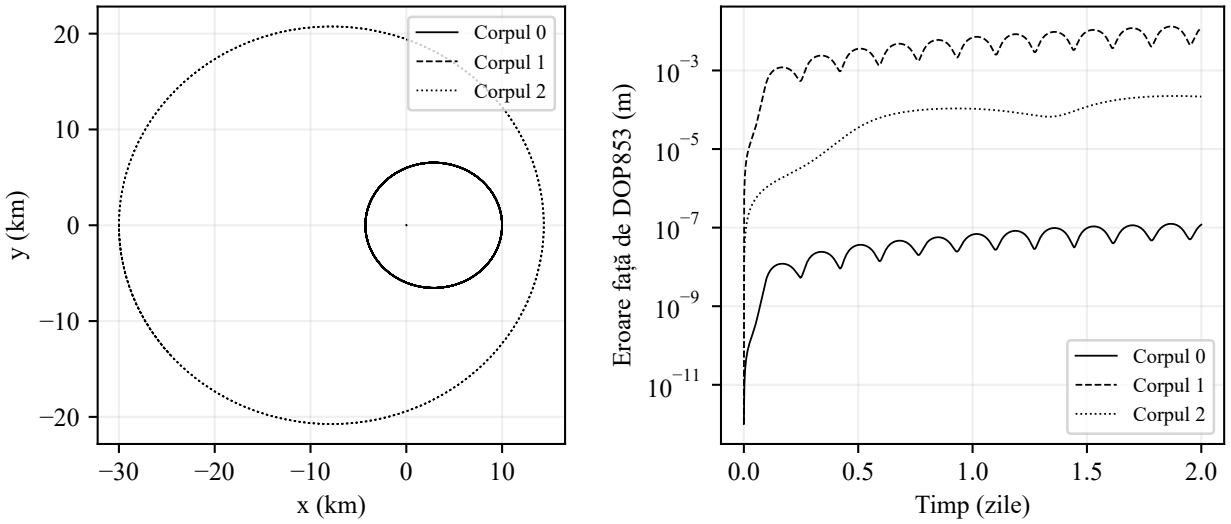


Figura 4: Cazul N3: traiectorii și diferențe de poziție față de DOP853. Referința include interacțiunile tuturor celor trei corpuri; nu este o superpoziție de două orbite kepleriene independente.

7.2 Cazul C4: unirea corpurilor

Experimentul simetric C4 pornește de la patru mase egale de 10^{15} kg, la $(\pm 3000, 0)$ și $(0, \pm 3000)$ m, cu viteze de 0,2 m/s orientate spre centru. Analiza arhivată a fișierului a înregistrat schimbarea numărului activ de la 4 la 1 la aproximativ 691,8 s, fără valori nefinite, și o poziție finală practic nulă. Ultimul cadru era la 4999,9 s, din cauza acumulării temporale.

Acest caz verifică funcționarea detectării contactului, reducerea numărului activ și compatibilitatea vizualizatorului cu fuziunile. Conservarea masei și a impulsului este proprietatea regulii (6); binarul pozițional singur nu măsoară aceste două mărimi. Energia de dinainte și de după fuziune nu constituie același test conservativ ca în K2.

8 Sistemul cu zece corpuri pe 100 de ani

8.1 Configurația și verificarea structurală

Cazul N10 cuprinde un corp de 10^{30} kg și nouă corpuri de 10^{27} kg; toate sunt mobile. Pozițiile inițiale ale corpurilor mici sunt între 10^{11} și $2{,}6 \times 10^{11}$ m de origine, cu viteze de ordinul 10^4 m/s. Durata este 3153600000 s, iar pasul este 3600 s. Sunt zece corpuri în total, nu zece planete plus un corp central suplimentar.

Fișierul are 150672192 octeți și conține 876001 cadre:

$$20 + 876001(12 + 16 \times 10) = 150672192. \tag{22}$$

Toate cadrele au zece corpuri, coordonate finite și timpi consecutivi separați cu 3600 s. Nu este înregistrată nicio fuziune. Figura 5 folosește 20374 stări pentru desen; validarea numărului de corpuri și a timpilor utilizează întregul fișier.

8.2 Reconstrucția vitezelor din pozițiile leapfrog

Deși binarul nu salvează vitezele, pentru o secvență fără fuziuni, cu pas fix și aceeași schemă (8)–(10), se poate deduce

$$\mathbf{v}_n = \frac{\mathbf{r}_{n+1} - \mathbf{r}_{n-1}}{2h}, \qquad \mathbf{v}_N = \frac{\mathbf{r}_N - \mathbf{r}_{N-1}}{h} + \frac{h}{2}\mathbf{a}(\mathbf{r}_N). \tag{23}$$

Prima identitate rezultă prin adunarea celor două viteze intermediare adiacente. Pentru stările schemei, ea este o identitate algebrică în aritmetică exactă, nu doar o aproximație generică de derivare. Aritmetica finită introduce totuși erori de scădere. Vitezele inițiale sunt preluate direct din intrare. Reconstrucția nu este aplicată peste fuziuni.

Energia N10 reconstruită reproduce valorile comunicate de autor la aproximativ 10^{-14} relativ pentru energia maximă:

$$E_0 \simeq -2{,}369268684338345 \times 10^{36}\ \text{J}, \tag{24}$$

$$E_{\max} \simeq 1{,}885250573897234 \times 10^{38}\ \text{J}, \tag{25}$$

$$\Delta E_{\max} \simeq 1{,}908943260740618 \times 10^{38}\ \text{J}, \tag{26}$$

$$\epsilon_{E,\max} \simeq 80{,}5709911 \quad (8057{,}09911\%). \tag{27}$$

Astfel, în acest experiment variația energiei depășește cu mult modulul energiei inițiale; nu poate fi descrisă ca o eroare mică de conservare.

8.3 Apropierea strânsă și interpretarea dinamicală

Cea mai mică distanță dintre centrele salvate este $23663637{,}47$ m, la 356558400 s, aproximativ 11,3064 ani. Perechea este formată din corpurile de indici 2 și 5, indexate de la zero. Suma razelor lor este 2×10^7 m, deci contactul nu este detectat în cadre. Distanța minimă dintre cadre poate fi diferită.

Primul timp salvat la care energia devine pozitivă coincide cu acest cadru. Scara locală $\tau \sim \sqrt{r^3/[G(m_i + m_j)]}$ este de aproximativ 315 s, față de pasul de 3600 s; raportul h/τ este aproximativ 11,4. Prin urmare, întâlnirea apropiată este parcursă cu un pas mare față de scara ei gravitațională. Coincidența temporală și saltul energetic susțin interpretarea unei întâlniri insuficient rezolvate numeric.

Traiectoriile ulterior extinse nu demonstrează, singure, ejecții fizice corecte sau haos determinist. Sensibilitatea haotică necesită o analiză de perturbații și convergență distinctă; aici mărimea abaterii energetice împiedică identificarea

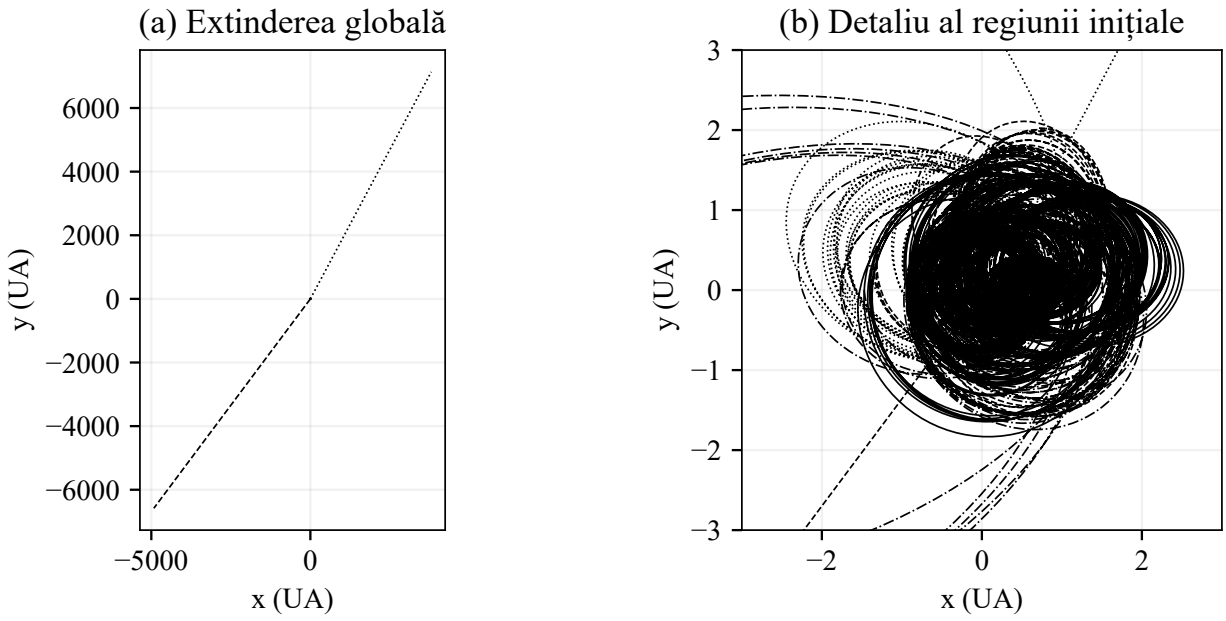


Figura 5: Cazul N10: vedere globală și detaliu al regiunii inițiale. Stilurile monocrome indică trasee distincte, dar figura globală este în principal un diagnostic al extinderii spațiale. UA reprezintă 149597870700 m.

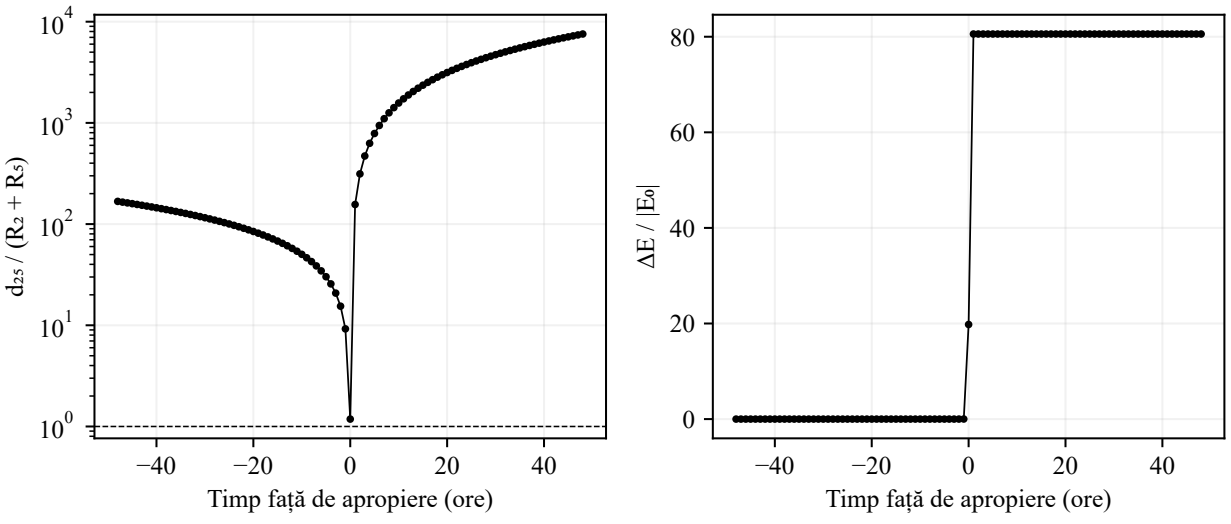


Figura 6: Detaliul întâlnirii N10 la 11,3064 ani: 97 de cadre orare, fără subeșantionare. Stânga: distanța dintre corpurile 2 și 5 raportată la suma razelor; linia întreruptă este pragul de contact. Dreapta: saltul energetic în aceeași fereastră temporală.

agitației vizuale cu fidelitatea față de modelul continuu. Aceasta este interpretarea rezultatului calculat, nu o modificare a configurației simulate.

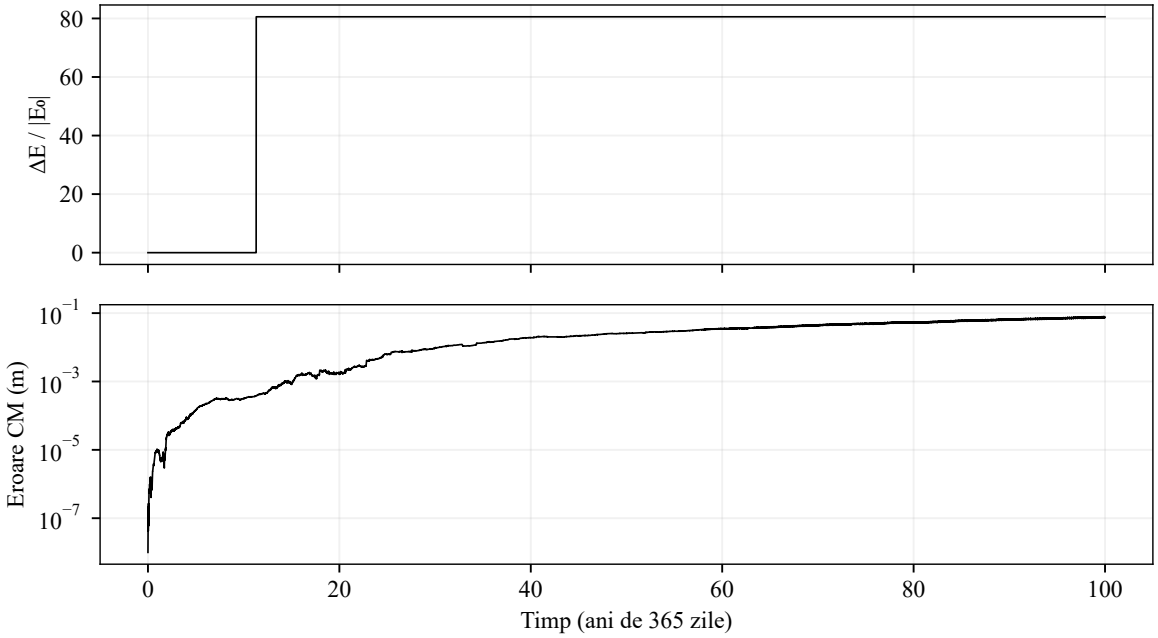


Figura 7: Energia N10 reconstruită și abaterea centrului de masă de la mișcarea rectilinie uniformă. Pe scara acestui experiment, impulsul global este mult mai bine păstrat decât energia.

Impulsul inițial N10 este $(-1{,}8 \times 10^{31},\ 3{,}95 \times 10^{31})$ kg m/s, deci centrul de masă nu este fix. Abaterea sa maximă eșantionată față de mișcarea teoretică este aproximativ 0,0796 m, în timp ce coordonatele orbitale sunt de ordin astronomic. Acest rezultat arată că un diagnostic global de impuls poate rămâne foarte bun chiar atunci când energia și evoluția orbitală sunt grav afectate.

9 Comparație între experimente și concluzii

Tabelul 3 păstrează valorile istorice de energie fără a le amesteca cu reluările K2. Seriile H1 și H2 au energia inițială comună $-61790801524{,}168$ J și provin dintr-o etapă anterioară cu observator și alte mase; seria H3 are $E_0 = -91886372572{,}5$ J. Aceste serii nu formează automat un singur studiu de convergență. În particular, schimbarea maselor schimbă problema fizică.

Tabelul 3: Valori energetice comunicate în istoricul experimentelor. Pașii care nu sunt documentați printr-un binar separat nu sunt atribuiți ca certitudini retrospective.

Serie	E_0 (J)	ΔE_+ (J)	ΔEmax (J)	$\epsilon_{E,\max}$
3 corpuri, seria H1	$-6{,}179 \times 10^{10}$	$4{,}747 \times 10^{1}$	$9{,}992 \times 10^{3}$	$1{,}617 \times 10^{-7}$
3 corpuri, seria H2	$-6{,}179 \times 10^{10}$	$4{,}848 \times 10^{-1}$	$9{,}992 \times 10^{1}$	$1{,}617 \times 10^{-9}$
3 corpuri, seria H3	$-9{,}189 \times 10^{10}$	$1{,}931 \times 10^{2}$	$4{,}079 \times 10^{4}$	$4{,}440 \times 10^{-7}$
2 corpuri, seria A	$-1{,}591 \times 10^{15}$	$4{,}374 \times 10^{7}$	$5{,}071 \times 10^{9}$	$3{,}187 \times 10^{-6}$
2 corpuri, seria B	$-1{,}591 \times 10^{15}$	$1{,}834 \times 10^{4}$	$2{,}028 \times 10^{6}$	$1{,}275 \times 10^{-9}$
10 corpuri, 100 ani	$-2{,}369 \times 10^{36}$	$1{,}909 \times 10^{38}$	$1{,}909 \times 10^{38}$	$8{,}057 \times 10^{1}$

Validarea cu două corpuri oferă o comparație directă cu soluția analitică, atât pentru energie, cât și pentru poziție la același timp. Pe intervalul studiat, erorile urmăresc aproape exact ordinul doi: pasul de 0,1 s produce erori individuale maxime eșantionate de ordinul a 0,1–0,2 mm. Curbele de fază explică de ce simpla suprapunere vizuală a orbitelor este insuficientă pentru această măsurare.

În sistemul cu trei corpuri, referința independentă confirmă diferențe de poziție de ordinul milimetrilor pentru corpul interior, fără a invoca o soluție analitică inexistentă pentru configurația generală. Experimentul cu fuziuni evidențiază schimbarea modelului energetic la contact. În sistemul cu zece corpuri, apropierea strânsă și variația energetică de 80,57

ori energia inițială în modul caracterizează o degradare numerică majoră a acelei rulări. Luate împreună, experimentele delimitează cantitativ regimul orbital validat și comportamentul observat în configurațiile mai dinamice.

Bibliografie

[1] E. Hairer, C. Lubich, G. Wanner, *Geometric numerical integration illustrated by the Störmer/Verlet method*, Acta Numerica 12 (2003), 399–450. Pagina autorilor: `https://www.unige.ch/~hairer/preprints/gniverlet.html`.

[2] Jet Propulsion Laboratory, *Fundamentals of Orbital Mechanics*, capitolul 7, ecuația Kepler și propagarea orbitală. `https://spsweb.fltops.jpl.nasa.gov/portaldataops/mpg/MPG_Docs/MPG%20Book/Release/Chapter7-OrbitalMechanics.pdf`.

[3] SciPy, *solve_ivp*: metoda DOP853, documentația oficială. `https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html`. Implementarea de analiză: SciPy 1.15.3.

A Datele de intrare și trasabilitatea calculelor

A.1 Două corpuri, pas de 0,1 s

```
2 172800 0.1
1e15 100 -5000 0 0 -0.65
5e14 80 10000 0 0 1.30
```

A.2 Trei corpuri, pas de 0,5 s

```
3 172800 0.5
1e15 100 0 0 0 0.000016
1e10 10 10000 0 0 2.0
3e10 15 -30000 0 0 -1.2
```

A.3 Patru corpuri, contact simetric

```
4 5000 0.1
1e15 100 -3000 0 0.2 0
1e15 100 3000 0 -0.2 0
1e15 100 0 -3000 0 0.2
1e15 100 0 3000 0 -0.2
```

A.4 Zece corpuri, 100 de ani

```
10 3153600000 3600
1e30 1e8 0 0 -10 22.5
1e27 1e7 1e11 0 0 22000
1e27 1e7 0 1.2e11 -21000 0
1e27 1e7 -1.4e11 0 0 -19000
1e27 1e7 0 -1.6e11 18000 0
1e27 1e7 1.8e11 0 -2000 15000
1e27 1e7 0 2e11 -15000 1000
1e27 1e7 -2.2e11 0 1000 -14000
1e27 1e7 0 -2.4e11 13000 -1000
1e27 1e7 2.6e11 0 -2000 13000
```

Metadatele, rezultatele și curbele de eroare sunt incluse în fișiere JSON și CSV alături de sursa LaTeX. Pentru K2, maximele energetice sunt măsurate la fiecare pas; maximele de poziție sunt eșantionate. Pentru N3, diferențele sunt calculate față de DOP853 la 17281 timpi. Pentru N10, energia este reconstruită din toate cadrele, iar graficele sunt eșantionate.

Amprenta SHA-256 a copiei binare N10 analizate este:

`1f6241cbb3817bd945cb00fa167004b6b0dbeae2d01fec4542e705cfa1e20e70`

Fișierul N10 are 150672192 octeți. Arhiva proiectului păstrează intrările, sursele și rezultatele prelucrate; nu dublează acest binar voluminos.

B Implementarea C++

B.1 Programul principal: main.cpp

```cpp
//g++ *.cpp -o main && ./main

#include "star.h"

#include <iomanip>


int main()
{
    const char* output = "simulation.bin";
    const char* input = "input.txt";

    ifstream in(input);

    std::cout << setprecision(17);

    int number_of_celestial_objects;
    double max_time, time_step;

    if (!(in >> number_of_celestial_objects >> max_time >> time_step)
        || number_of_celestial_objects <= 0
        || max_time < 0
        || time_step <= 0)
    {
        cerr << "invalid simulation settings\n";
        return 1;
    }

    ofstream out(output, ios::binary | ios::trunc);

    if (!out)
    {
        cerr << "could not open " << output << endl;
        return 1;
    }

    out.write(
        reinterpret_cast<const char*>(&number_of_celestial_objects),
        sizeof(number_of_celestial_objects)
    );

    out.write(
        reinterpret_cast<const char*>(&max_time),
        sizeof(max_time)
    );

    out.write(
        reinterpret_cast<const char*>(&time_step),
        sizeof(time_step)
    );

    celestial_object* celestial_objects =
        new celestial_object[number_of_celestial_objects];

    for (int i = 0; i < number_of_celestial_objects; i++)
    {
        if (!(in >> celestial_objects[i]))
        {
            cerr << "could not read object " << i << '\n';
            delete[] celestial_objects;
            return 1;
        }
    }

    in.close();

    vector_2d* forces = new vector_2d[number_of_celestial_objects];

    const double initial_energy =
        total_energy(celestial_objects, number_of_celestial_objects);
```

```
71
72      double max_energy = initial_energy;
73      double max_error = 0.0;
74
75      for (double time = 0; time <= max_time; time += time_step)
76      {
77          set_force(
78              forces, celestial_objects, number_of_celestial_objects
79          );
80
81          double current_energy =
82              total_energy(celestial_objects, number_of_celestial_objects);
83
84          if (current_energy > max_energy)
85              max_energy = current_energy;
86
87          if (abs(current_energy - initial_energy) > max_error)
88              max_error = abs(current_energy - initial_energy);
89
90          // Save time and current body count.
91          out.write(
92              reinterpret_cast<const char*>(&time),
93              sizeof(time)
94          );
95
96          out.write(
97              reinterpret_cast<const char*>(&number_of_celestial_objects),
98              sizeof(number_of_celestial_objects)
99          );
100
101         // Save positions before updating any bodies.
102         for (int j = 0; j < number_of_celestial_objects; j++)
103         {
104             const vector_2d position =
105                 celestial_objects[j].get_position();
106
107             out.write(
108                 reinterpret_cast<const char*>(&position.x),
109                 sizeof(double)
110             );
111
112             out.write(
113                 reinterpret_cast<const char*>(&position.y),
114                 sizeof(double)
115             );
116         }
117
118         if (!out)
119         {
120             cerr << "could not write simulation step\n";
121             delete[] celestial_objects;
122             delete[] forces;
123             return 1;
124         }
125
126         // First velocity half-step.
127         for (int j = 0; j < number_of_celestial_objects; j++)
128         {
129             update_velocity(
130                 forces[j], celestial_objects[j], time_step / 2
131             );
132         }
133
134         // Position full-step.
135         for (int j = 0; j < number_of_celestial_objects; j++)
136         {
137             update_position(celestial_objects[j], time_step);
138         }
139
140         set_force(
141             forces, celestial_objects, number_of_celestial_objects
142         );
143
144         // Second velocity half-step.
145         for (int j = 0; j < number_of_celestial_objects; j++)
```

```
146         {
147             update_velocity(
148                 forces[j], celestial_objects[j], time_step / 2
149             );
150         }
151 
152         if ((int)time % 100000 == 0)
153         {
154             std::cout << 100 * time / max_time
155                       << "% completed" << endl;
156         }
157     }
158 
159     delete[] celestial_objects;
160     delete[] forces;
161 
162     out.close();
163 
164     if (!out)
165     {
166         cerr << "could not finish writing output\n";
167         return 1;
168     }
169 
170     std::cout << "initial energy: " << initial_energy
171               << " maximum energy: " << max_energy << endl;
172 
173     std::cout << "energy error: "
174               << abs(initial_energy - max_energy) << endl;
175 
176     std::cout << "max energy error: " << max_error << endl;
177 
178     return 1;
179 }
```

B.2 Interfețele: star.h

```
#include <iostream>
#include <fstream>
#include <cmath>

#define G 6.67430e-11

using namespace std;

class vector_2d
{

    public:

    double x = 0;
    double y = 0;

    vector_2d() = default;

    vector_2d(double x, double y);

    //getters

    //calcul operators
    vector_2d operator+(const vector_2d& AUX) const;
    vector_2d operator*(const vector_2d& AUX) const;
    vector_2d operator*(const double AUX) const;

    //IO operators
    friend istream& operator>>(istream& input, vector_2d& vector);
    friend ostream& operator<<(ostream& output, const vector_2d& vector);

};

class celestial_object
{

    private:

    //mass, radius, position, velocity, acceleration:

    double mass = 0; //kg
    double radius = 0; //m

    vector_2d position{0, 0}; //m
    vector_2d velocity{0, 0}; //m/s
    vector_2d acceleration{0, 0}; //m/s^2

    public:

    celestial_object() = default;

    celestial_object(double mass, double radius, double px, double py, double vx, double vy);

    //getters
    double get_mass() const;
    double get_radius() const;

    vector_2d get_position() const;
    vector_2d get_velocity() const;
    vector_2d get_acceleration() const;

    //setters
    void set_mass(const double& AUX);
    void set_radius(const double& AUX);

    void set_position(const vector_2d& AUX);
    void set_velocity(const vector_2d& AUX);
    void set_acceleration(const vector_2d& AUX);

    //calcul operators

    celestial_object operator+(const celestial_object& AUX) const;
```

```
73
74      //IO operators
75      friend istream& operator>>(istream& input, celestial_object& AUX);
76
77  };
78
79  class force_ij
80  {
81
82      private:
83
84          vector_2d component{0, 0};
85
86          double modulus = 0;
87          double distance = 0;
88
89      public:
90
91      force_ij() = default;
92
93      //getters
94
95      vector_2d get_component();
96
97      double get_modulus();
98      double get_distance();
99
100     //setters
101
102     void set_component(const vector_2d& AUX);
103
104     void set_modulus(const double& AUX);
105     void set_distance(const double& AUX);
106
107 };
108
109 void merge_celestial_objects( celestial_object* objects, int* count, int A, int B);
110
111 void resolve_collisions(celestial_object* objects, int& count);
112
113 void set_force( vector_2d* forces, celestial_object* objects, int& count);
114
115 void update_velocity(const vector_2d AUX, celestial_object& object, double time_step);
116
117 void update_position(celestial_object& object, double time_step);
118
119 double total_energy(const celestial_object* objects, int count);
120
121 int write(double time, const celestial_object& A, ostream& output);
```

B.3 Modelul și integratorul: star.cpp

```cpp
#include "star.h"

vector_2d :: vector_2d(double x, double y)
{

    this->x = x;
    this->y = y;

}

vector_2d vector_2d :: operator+(const vector_2d& AUX) const { return vector_2d(x + AUX.x, y + AUX.y); }
vector_2d vector_2d :: operator*(const vector_2d& AUX) const { return vector_2d(x * AUX.x, y * AUX.y); }
vector_2d vector_2d :: operator*(const double AUX) const  { return vector_2d(x * AUX, y * AUX); }

//IO operators
istream& operator>>(istream& input, vector_2d& vector){ input >> vector.x >> vector.y; return input; }
ostream& operator<<(ostream& output, const vector_2d& vector){ output << vector.x << " " << vector.y; return
     output; }

celestial_object :: celestial_object(double mass, double radius, double px, double py, double vx, double vy)
{

    this->mass = mass;
    this->radius = radius;

    position.x = px; position.y = py;
    velocity.x = vx; velocity.y = vy;

}

//getters

double celestial_object :: get_mass() const { return mass; }
double celestial_object :: get_radius() const { return radius; }

vector_2d celestial_object :: get_position() const { return position; }
vector_2d celestial_object :: get_velocity() const { return velocity; }
vector_2d celestial_object :: get_acceleration() const { return acceleration; }

//setters

void celestial_object :: set_mass(const double& AUX) { mass = AUX; }
void celestial_object :: set_radius(const double& AUX) { radius = AUX; }

void celestial_object :: set_position(const vector_2d& AUX) { position = AUX; }
void celestial_object :: set_velocity(const vector_2d& AUX) { velocity = AUX; }
void celestial_object :: set_acceleration(const vector_2d& AUX) { acceleration = AUX; }

celestial_object celestial_object :: operator+(const celestial_object& AUX) const
{

    return celestial_object(mass + AUX.mass, radius + AUX.radius,
                            position.x + AUX.position.x, position.y + AUX.position.y,
                            velocity.x + AUX.velocity.x, velocity.y + AUX.velocity.y);
}

istream& operator>>(istream& input, celestial_object& AUX)
{

    input >> AUX.mass >> AUX.radius

    >> AUX.position.x >> AUX.position.y
    >> AUX.velocity.x >> AUX.velocity.y;

    AUX.set_acceleration(vector_2d(0, 0));

    return input;

}

vector_2d force_ij :: get_component(){ return vector_2d(component.x, component.y); }
double force_ij :: get_modulus(){ return modulus; }
```

```cpp
double force_ij :: get_distance(){ return distance; }

void force_ij :: set_component(const vector_2d& AUX) { component = AUX; }
void force_ij :: set_modulus(const double& AUX) { modulus = AUX; }
void force_ij :: set_distance(const double& AUX) { distance = AUX; }

void merge_celestial_objects( celestial_object* objects, int* count, int A, int B)
{

    const double ma = objects[A].get_mass();
    const double mb = objects[B].get_mass();
    const double mass = ma + mb;

    const vector_2d position =
        (objects[A].get_position() * ma
       + objects[B].get_position() * mb) * (1.0 / mass);

    const vector_2d velocity =
        (objects[A].get_velocity() * ma
       + objects[B].get_velocity() * mb) * (1.0 / mass);

    const double ra = objects[A].get_radius();
    const double rb = objects[B].get_radius();

    const double radius =
        std::cbrt(ra * ra * ra + rb * rb * rb);

    objects[A] = celestial_object(
        mass, radius,
        position.x, position.y,
        velocity.x, velocity.y
    );

    for (int k = B; k < *count - 1; ++k)
        objects[k] = objects[k + 1];

    --(*count);
}

void resolve_collisions(celestial_object* objects, int& count)
{
    bool merged;

    do
    {
        merged = false;

        for (int i = 0; i < count - 1 && !merged; ++i)
        {
            for (int j = i + 1; j < count; ++j)
            {
                const vector_2d a = objects[i].get_position();
                const vector_2d b = objects[j].get_position();

                const double distance =
                    std::hypot(b.x - a.x, b.y - a.y);

                const double total_radius =
                    objects[i].get_radius() + objects[j].get_radius();

                if (distance <= total_radius)
                {
                    merge_celestial_objects(objects, &count, i, j);
                    merged = true;
                    break;
                }
            }
        }
    }
    while (merged);
}

void set_force( vector_2d* forces, celestial_object* objects, int& count)
{
    resolve_collisions(objects, count);
```

```
147
148     for (int i = 0; i < count; ++i)
149         forces[i] = vector_2d(0, 0);
150
151     for (int i = 0; i < count - 1; ++i)
152     {
153         for (int j = i + 1; j < count; ++j)
154         {
155             const vector_2d a = objects[i].get_position();
156             const vector_2d b = objects[j].get_position();
157
158             const vector_2d displacement(b.x - a.x, b.y - a.y);
159             const double distance =
160                 std::hypot(displacement.x, displacement.y);
161
162             if (distance == 0.0)
163                 continue;
164
165             const double magnitude =
166                 G * objects[i].get_mass() * objects[j].get_mass()
167                 / distance / distance;
168
169             const vector_2d force =
170                 displacement * (magnitude / distance);
171
172             forces[i] = forces[i] + force;
173             forces[j] = forces[j] + force * (-1.0);
174         }
175     }
176 }
177
178 void update_velocity(const vector_2d AUX, celestial_object& object, double time_step)
179 {
180
181     object.set_acceleration(vector_2d(AUX.x / object.get_mass(), AUX.y / object.get_mass()));
182
183     object.set_velocity(vector_2d(object.get_velocity().x + object.get_acceleration().x * time_step,
184                                   object.get_velocity().y + object.get_acceleration().y * time_step));
185
186 }
187
188 void update_position(celestial_object& object, double time_step)
189 {
190
191     object.set_position(vector_2d(object.get_position().x + object.get_velocity().x * time_step,
192                                   object.get_position().y + object.get_velocity().y * time_step));
193
194 }
195
196 double total_energy(const celestial_object* objects, int count)
197 {
198     double energy = 0.0;
199
200     for (int i = 0; i < count; ++i)
201     {
202         const vector_2d velocity = objects[i].get_velocity();
203
204         energy += 0.5 * objects[i].get_mass()
205                 * (velocity.x * velocity.x + velocity.y * velocity.y);
206
207         for (int j = i + 1; j < count; ++j)
208         {
209             const vector_2d a = objects[i].get_position();
210             const vector_2d b = objects[j].get_position();
211
212             const double distance = std::hypot(b.x - a.x, b.y - a.y);
213
214             // Requires distinct centers: distance > 0.
215             energy -= G * objects[i].get_mass()
216                         * objects[j].get_mass() / distance;
217         }
218     }
219
220     return energy;
221 }
```

```
222
223 int write(double time, const celestial_object& A, ostream& output)
224 {
225     static_assert(
226         is_trivially_copyable_v<celestial_object>,
227         "object requires custom serialization"
228     );
229
230     output.write(reinterpret_cast<const char*>(&time), sizeof(time));
231
232     const vector_2d position = A.get_position();
233
234     output.write(reinterpret_cast<const char*>(&position), sizeof(position));
235
236     if (!output)
237     {
238         std::cerr << "could not write simulation record\n";
239         return 1;
240     }
241
242     return 0;
243 }
```

C Vizualizatorul Python

```python
import argparse
import struct
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import Button, Slider, TextBox


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('file', nargs='?', default='simulation.bin')
    parser.add_argument('--save', help='Save a static image instead of opening the viewer')
    args = parser.parse_args()
    path = Path(args.file)
    with path.open('rb') as stream:
        header = stream.read(20)
    if len(header) != 20:
        parser.error('Incomplete header')
    count, duration, step = struct.unpack('<idd', header)
    if count < 1 or not np.isfinite([duration, step]).all() or step <= 0:
        parser.error('Invalid header')
    frames_data = []
    counts = []
    with path.open('rb') as stream:
        stream.seek(20)
        previous = -np.inf
        while True:
            frame_header = stream.read(12)
            if not frame_header:
                break
            if len(frame_header) != 12:
                parser.error('Incomplete step header')
            time, active = struct.unpack('<di', frame_header)
            if not np.isfinite(time) or time <= previous or not 1 <= active <= count:
                parser.error('Invalid step; regenerate the binary with the new output block')
            if counts and active > counts[-1]:
                parser.error('Body count increased unexpectedly')
            payload = stream.read(active * 16)
            if len(payload) != active * 16:
                parser.error('Incomplete positions')
            xy = np.frombuffer(payload, dtype='<f8').reshape(active, 2)
            if not np.isfinite(xy).all():
                parser.error('Nonfinite positions')
            row = np.full((count, 3), np.nan)
            row[:, 0] = time
            row[:active, 1:3] = xy
            frames_data.append(row)
            counts.append(active)
            previous = time
    if not frames_data:
        parser.error('No saved steps')
    records = np.asarray(frames_data)
    del frames_data
    counts = np.asarray(counts)
    frames = len(records)
    times = records[:, 0, 0]
    low = np.nanmin(records[:, :, 1:3], axis=(0, 1))
    high = np.nanmax(records[:, :, 1:3], axis=(0, 1))
    changes = np.flatnonzero(np.r_[False, counts[1:] != counts[:-1]])

    fig, ax = plt.subplots(figsize=(10, 8))
    fig.subplots_adjust(bottom=.25, top=.9)
    center = (low + high) / 2
    span = max(float((high - low).max()), 1.0)
    ax.set(xlim=(center[0]-.55*span, center[0]+.55*span),
           ylim=(center[1]-.55*span, center[1]+.55*span),
           xlabel='X (meters)', ylabel='Y (meters)')
    ax.set_aspect('equal')
    ax.grid(True, alpha=.25)
    trails, markers = [], []
    for body in range(count):
        line, = ax.plot([], [], lw=1, label=f'Body {body}')
```

```
        marker, = ax.plot([], [], 'o', color=line.get_color(), markersize=7)
        trails.append(line)
        markers.append(marker)
    ax.legend()
    title = ax.set_title('')
    status = fig.text(.12, .02, '', fontsize=9)
    state = {'index': 0, 'playing': False, 'updating': False}
    slider = Slider(fig.add_axes([.16, .15, .69, .03]), 'Time (s)',
                    float(times[0]), float(times[-1]) if frames > 1 else float(times[0])+1,
                    valinit=float(times[0]))
    button = Button(fig.add_axes([.12, .065, .14, .05]), 'Play')
    back = Button(fig.add_axes([.28, .065, .10, .05]), 'Previous')
    forward = Button(fig.add_axes([.40, .065, .10, .05]), 'Next')
    box = TextBox(fig.add_axes([.68, .065, .18, .05]), 'Jump to (s) ', initial='0')

    def pause():
        state['playing'] = False
        button.label.set_text('Play')

    def nearest(value):
        k = int(np.searchsorted(times, value))
        if k == 0:
            return 0
        if k >= frames:
            return frames - 1
        return k if times[k]-value < value-times[k-1] else k-1

    def show(index):
        state['index'] = int(index)
        # Limit trail rendering, but always show the exact selected saved state.
        # Indices may change after merging. Start new trails after each count change.
        history_start = int(changes[np.searchsorted(changes, index, side='right')-1]) if np.searchsorted(
 changes, index, side='right') else 0
        stride = max(1, (index-history_start+1)//10000)
        indices = np.unique(np.r_[np.arange(history_start, index+1, stride), index])
        xy = np.asarray(records[indices, :, 1:3])
        for body in range(count):
            trails[body].set_data(xy[:, body, 0], xy[:, body, 1])
            markers[body].set_data([records[index, body, 1]], [records[index, body, 2]])
        title.set_text(f'Time: {times[index]:,.6f} s | Bodies: {counts[index]} | Step {index+1:,}/{frames:,}')
        state['updating'] = True
        try:
            slider.set_val(float(times[index]))
        finally:
            state['updating'] = False
        fig.canvas.draw_idle()

    def seek(value):
        if state['updating']:
            return
        pause()
        show(nearest(value))
        status.set_text('Paused at nearest saved time. Space: play/pause; arrows: one saved step.')

    def jump(text):
        try:
            value = float(text)
            if not np.isfinite(value):
                raise ValueError
        except ValueError:
            status.set_text('Enter a finite time in seconds.')
            fig.canvas.draw_idle()
            return
        pause()
        show(nearest(value))
        status.set_text(f'Requested {value:g} s; showing nearest saved time {times[state["index"]]:g} s.')
        fig.canvas.draw_idle()

    def toggle(event=None):
        if not state['playing'] and state['index'] == frames-1:
            show(0)
        state['playing'] = not state['playing']
        button.label.set_text('Pause' if state['playing'] else 'Play')
        fig.canvas.draw_idle()

```

```
    def move(amount):
        pause()
        show(max(0, min(frames-1, state['index']+amount)))

    def key(event):
        if event.inaxes is box.ax:
            return
        if event.key == ' ':
            toggle()
        elif event.key == 'right':
            move(1)
        elif event.key == 'left':
            move(-1)

    def tick():
        if state['playing']:
            show(min(frames-1, state['index']+max(1, (frames-1)//1800)))
            if state['index'] == frames-1:
                pause()
                fig.canvas.draw_idle()

    slider.on_changed(seek)
    box.on_submit(jump)
    button.on_clicked(toggle)
    back.on_clicked(lambda event: move(-1))
    forward.on_clicked(lambda event: move(1))
    fig.canvas.mpl_connect('key_press_event', key)
    timer = fig.canvas.new_timer(interval=33)
    timer.add_callback(tick)
    show(0)
    status.set_text('Space: play/pause. Arrows: one saved step. Jump uses nearest saved time.')
    print(f'{frames:,} saved steps, {count} bodies; {times[0]:g} to {times[-1]:g} seconds.')
    if args.save:
        show(frames-1)
        fig.savefig(args.save, dpi=160)
        plt.close(fig)
    else:
        timer.start()
        plt.show()
        timer.stop()


if __name__ == '__main__':
    main()
```

D Programul de măsurare pentru reluările K2

Acest program apelează funcțiile din sursa analizată și separă energia măsurată la fiecare pas de stările eșantionate pentru comparația analitică. Argumentele sunt: intrarea, ieșirea CSV, intervalul de salvare exprimat în pași și ieșirea JSON.

```
#include "src/star.h"
#include <algorithm>
#include <cstdlib>
#include <iomanip>
#include <type_traits>
#include <vector>
int main(int argc, char **argv) {
  if (argc != 5)
    return 2;
  ifstream in(argv[1]);
  int n;
  double T, h;
  in >> n >> T >> h;
  vector<celestial_object> b(n);
  for (auto &x : b)
    in >> x;
  vector<vector_2d> F(n);
  ofstream out(argv[2]);
  out << setprecision(17);
  int stride = atoi(argv[3]);
  double E0 = total_energy(b.data(), n), maxE = E0, maxAbs = 0, minE = E0;
  double minD = 1e300;
  long long k = 0;
  double last = 0;
  out << "t,E";
  for (int i = 0; i < n; i++)
    out << ",x" << i << ",y" << i << ",vx" << i << ",vy" << i;
  out << "\n";
  for (double t = 0; t <= T; t += h, ++k) {
    set_force(F.data(), b.data(), n);
    double E = total_energy(b.data(), n);
    maxE = max(maxE, E);
    minE = min(minE, E);
    maxAbs = max(maxAbs, abs(E - E0));
    last = t;
    for (int i = 0; i < n; i++)
      for (int j = i + 1; j < n; j++) {
        auto x = b[i].get_position(), y = b[j].get_position();
        minD = min(minD, hypot(x.x - y.x, x.y - y.y));
      }
    if (k % stride == 0 || t + h > T) {
      out << t << "," << E;
      for (int i = 0; i < n; i++) {
        auto r = b[i].get_position(), v = b[i].get_velocity();
        out << "," << r.x << "," << r.y << "," << v.x << "," << v.y;
      }
      out << "\n";
    }
    for (int i = 0; i < n; i++)
      update_velocity(F[i], b[i], h / 2);
    for (int i = 0; i < n; i++)
      update_position(b[i], h);
    set_force(F.data(), b.data(), n);
    for (int i = 0; i < n; i++)
      update_velocity(F[i], b[i], h / 2);
  }
  ofstream s(argv[4]);
  s << setprecision(17) << "{\"h\":" << h << ",\"T\":" << T << ",\"E0\":" << E0
    << ",\"Emax\":" << maxE << ",\"Emin\":" << minE << ",\"max_abs\":" << maxAbs
    << ",\"positive\":" << maxE - E0 << ",\"frames\":" << k
    << ",\"last\":" << last << ",\"min_distance\":" << minD
    << ",\"active\":" << n << "}";
}
```